

FileInfo Class in C# with Examples

In this article, I will discuss **FileInfo Class in C# with Examples**. Please read our previous article discussing [StringWriter and StringReader in C# with Examples](#). At the end of this article, you will understand what FileInfo class is in C# and when and how to use FileInfo Class in C# with Examples.

What is FileInfo Class in C#?

The FileInfo class in C# is used for manipulating files such as creating, deleting, removing, copying, opening, and getting information. The FileInfo class provides several properties and methods that make file manipulation easy. FileInfo class is used for typical file operations like copying, moving, renaming, creating, opening, deleting, and appending files. By default, full read/write access to new files is granted to all users.

The FileInfo class in C# belongs to the System.IO namespace. It is a sealed class, and hence, it cannot be inherited. If you go to the definition of the FileInfo class, you will see the following.

```
namespace System.IO
{
    public sealed class FileInfo : FileSystemInfo
    {
        public FileInfo(string fileName);

        public DirectoryInfo Directory { get; }
        public string DirectoryName { get; }
        public long Length { get; }
        public override string Name { get; }
        public bool IsReadOnly { get; set; }
        public override bool Exists { get; }

        public StreamWriter AppendText();
        public FileInfo CopyTo(string destFileName);
        public FileInfo CopyTo(string destFileName, bool overwrite);
        public FileStream Create();
        public StreamWriter CreateText();
        public void Decrypt();
        public override void Delete();
        public void Encrypt();
        public FileSecurity GetAccessControl();
        public FileSecurity GetAccessControl(AccessControlSections includeSections);
        public void MoveTo(string destFileName);
        public FileStream Open(FileMode mode);
        public FileStream Open(FileMode mode, FileAccess access, FileShare share);
        public FileStream Open(FileMode mode, FileAccess access);
        public FileStream OpenRead();
        public StreamReader OpenText();
        public FileStream OpenWrite();
        public FileInfo Replace(string destinationFileName, string destinationBackupFileName);
        public FileInfo Replace(string destinationFileName, string destinationBackupFileName, bool ignoreMetadataErrors);
        public void SetAccessControl(FileSecurity fileSecurity);
        public override string ToString();
    }
}
```

The FileInfo class in C# provides the following Constructor, Methods, and Properties to work with files.

The Constructor of FileInfo Class in C#

The FileInfo Class provides the following Constructor

public FileInfo(string fileName): It initializes a new instance of the System.IO.FileInfo class, which acts as a wrapper for a file path. The parameter fileName specifies the fully qualified name of the new file or the relative file name. Do not end the path with the directory separator character.

Properties of FileInfo Class in C#

The FileInfo Class provides the following properties.

1. **Directory:** It is used to get an instance of the parent directory. It returns a DirectoryInfo object representing the parent directory of this file.
2. **DirectoryName:** It gets a string representing the directory's full path. It returns a string representing the directory's full path.
3. **Length:** It is used to get the size, in bytes, of the current file. It returns the size of the current file in bytes.
4. **Name:** It is used to get the name of the file.
5. **IsReadOnly:** It is used to get or set a value determining whether the current file is read-only. It returns true if the current file is read-only; otherwise, false.
6. **Exists:** It is used to get a value indicating whether a file exists. It returns true if the file exists, false if it does not exist, or if it is a directory.

FileInfo Class Methods in C#

The FileInfo class in C# provides the following methods.

1. **public StreamWriter AppendText():** This method is used to get a value indicating whether a file exists. It returns true if the file exists, false if it does not exist, or if it is a directory.
2. **public FileInfo CopyTo(string destFileName):** This method is used to copy an existing file to a new one, disallowing the overwriting of an existing one. It returns a new file with a fully qualified path. The parameter destFileName specifies the name of the new file to copy to.
3. **public FileInfo CopyTo(string destFileName, bool overwrite):** This method is used to copy an existing file to a new one, allowing the overwriting of an existing one. It returns a new file or an overwrite of an existing file if overwrite is true. If the file exists and overwrite is false, an IOException is thrown. The parameter destFileName specifies the name of the new file to copy to, and the parameter overwrites specifies true to allow an existing file to be overwritten; otherwise, false.
4. **public FileStream Create():** This method creates and returns a new file.
5. **public StreamWriter CreateText():** This method creates a StreamWriter that writes a new text file.
6. **public void Decrypt():** This method is used to decrypt a file encrypted by the current account using the System.IO.FileInfo.Encrypt method.
7. **public override void Delete():** This method permanently deletes a file.
8. **public void Encrypt():** This method is used to encrypt a file so that only the account used to encrypt the file can decrypt it.
9. **public FileSecurity GetAccessControl():** This method is used to get a System.Security.AccessControl.FileSecurity object that encapsulates the access control list (ACL) entries for the file described by the current System.IO.FileInfo object. That means this method returns a System.Security.AccessControl.FileSecurity object that encapsulates the access control rules for the current file.
10. **public void MoveTo(string destFileName):** This method moves a specified file to a new location, providing the option to specify a new file name. Here, the destFileName parameter specifies the path to move the file to, which can specify a different file name.
11. **public FileStream Open(FileMode mode):** This method opens a file in the specified mode. It returns a file opened in the specified mode, with read/write access and unshared

12. **public FileStream Open(FileMode mode, FileAccess access):** This method is used to open a file in the specified mode with read, write, or read/write access. It returns a System.IO.FileStream object opened in the specified mode, accessed, and unshared.
13. **public FileStream Open(FileMode mode, FileAccess access, FileShare share):** This method opens a file in the specified mode with read, write, or read/write access and the specified sharing option. It returns a FileStream object opened with the specified mode, access, and sharing options. Here, the parameter mode specifies a System.IO.FileMode constant specifying the mode (for example, Open or Append) in which to open the file. The parameter access specifies a System.IO.FileAccess constant specifying whether to open the file with Read, Write, or ReadWrite file access, and the parameter share specifies a System.IO.FileShare constant specifying the type of access other FileStream objects have to this file.
14. **public FileStream OpenRead():** This method creates and returns a new read-only System.IO.FileStream.
15. **public StreamReader OpenText():** This method creates System.IO.StreamReader with UTF8 encoding that reads from an existing text file. It returns a new StreamReader with UTF8 encoding.
16. **public FileStream OpenWrite():** This method creates a write-only System.IO.FileStream. It returns a write-only unshared System.IO.FileStream object for a new or existing file.
17. **public FileInfo Replace(string destinationFileName, string destinationBackupFileName):** This method is used to replace the contents of a specified file with the file described by the current System.IO.FileInfo object, deleting the original file and creating a backup of the replaced file. It returns a System.IO.FileInfo object that encapsulates information about the file described by the destFileName parameter.
18. **public void SetAccessControl(FileSecurity fileSecurity):** This method is used to apply access control list (ACL) entries described by a System.Security.AccessControl.FileSecurity object to the file described by the current System.IO.FileInfo object.
19. **public override string ToString():** This method returns the path as a string.

Create a File in C# using FileInfo:

The Create method of the FileInfo class is used to create the new file. For a better understanding, please have a look at the below code. In the below code, we call the Create method on the instance of the FileInfo class, which will create the MyTestFile1.txt file in the D drive.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string path = @"D:\MyTestFile1.txt";
            FileInfo fileInfo = new FileInfo(path);
            fileInfo.Create();
            {
                Console.WriteLine("File has been created");
            }
            Console.ReadKey();
        }
    }
}
```

FileInfo Class CreateText Method in C#:

This method creates a StreamWriter that writes a new text file. For a better understanding, please have a look at the below code.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string path = @"D:\MyTestFile2.txt";
            FileInfo fileInfo = new FileInfo(path);
            StreamWriter str = fileInfo.CreateText();
            str.WriteLine("Hello");
            Console.WriteLine("File has been created with text");
            str.Close();
            Console.ReadKey();
        }
    }
}
```

FileInfo Class Delete Method in C#:

This method is used to delete an existing file. For a better understanding, please have a look at the below code.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string path = @"D:\MyTestFile2.txt";
            FileInfo fileInfo = new FileInfo(path);
            fileInfo.Delete();
            Console.WriteLine("File has been deleted");
            Console.ReadKey();
        }
    }
}
```

FileInfo Class CopyTo Method in C#:

This method is used to copy an existing file into a new file. For a better understanding, please have a look at the below code. Here, MyTestFile1.txt should exist in the D drive, and MyTestFile2.txt should not exist in the D drive; otherwise, it will not work.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string path1 = @"D:\MyTestFile1.txt";
            string path2 = @"D:\MyTestFile2.txt";

            FileInfo fileInfo1 = new FileInfo(path1);
            FileInfo fileInfo2 = new FileInfo(path2);
            fileInfo1.CopyTo(path2);

            Console.WriteLine("{0} was copied to {1}.", path1, path2);
            Console.ReadKey();
        }
    }
}
```

FileInfo Class MoveTo Method in C#:

This method moves the file from one place to another valid location. For a better understanding, please have a look at the below code. Here, sourcePath must exist, and the Dotnet folder should be there; otherwise, it will not work.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string sourcePath = @"D:\MyTestFile1.txt";
            string destinationPath = @"D:\Dotnet\MyTestFile1.txt";
            FileInfo fileInfo = new FileInfo(sourcePath);
            fileInfo.MoveTo(destinationPath);
            Console.WriteLine("{0} was moved to {1}.", sourcePath, destinationPath);
            Console.ReadKey();
        }
    }
}
```

FileInfo Class AppendText Method in C#:

The FileInfo Class AppendText Method in C# creates a StreamWriter that appends text to the file represented by this instance of the FileInfo. For a better understanding, please have a look at the below code.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string Path = @"D:\MyTestFile1.txt";
            FileInfo fileInfo = new FileInfo(Path);
            StreamWriter streamWriter = fileInfo.AppendText();
            streamWriter.WriteLine("This");
            streamWriter.WriteLine("is Extra");
            streamWriter.WriteLine("Text");
            Console.WriteLine("File has been appended");
            streamWriter.Close();
            Console.ReadKey();
        }
    }
}
```

FileInfo Class OpenText Method in C#:

The FileInfo Class OpenText Method in C# creates a StreamReader with UTF8 encoding that reads from an existing text file. For a better understanding, please have a look at the below code.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string Path = @"D:\MyTestFile1.txt";
            FileInfo fileInfo = new FileInfo(Path);
            StreamReader streamReader = fileInfo.OpenText();
            string s = "";
            while ((s = streamReader.ReadLine()) != null)
            {
                Console.WriteLine(s);
            }

            Console.ReadKey();
        }
    }
}
```

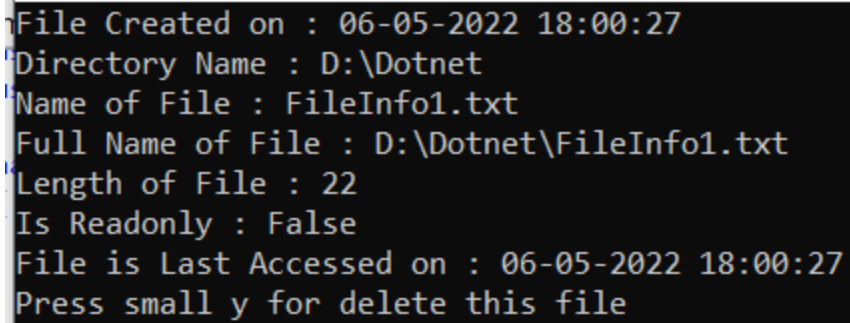
How to Get File Info in C#?

In the below example, I will show you the use of the FileInfo class in C# to perform different types of file operations. In the below example, we will search for the file **D:\Dotnet\FileInfo1.txt** file. If the file is found, then it displays the information of the file else, create a new file. But you should have a folder named Dotnet in the D drive. Otherwise, this program will not work.

```
using System;
using System.IO;
namespace FileInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string filePath = @"D:\Dotnet\FileInfo1.txt";
            FileInfo fileInfo = new FileInfo(filePath);
            //Create File
            using (StreamWriter sw = fileInfo.CreateText())
            {
                sw.WriteLine("Hello FileInfo Class");
            }
            //Display File Info
            Console.WriteLine("File Created on : " + fileInfo.CreationTime);
            Console.WriteLine("Directory Name : " + fileInfo.DirectoryName);
            Console.WriteLine("Name of File : " + fileInfo.Name);
            Console.WriteLine("Full Name of File : " + fileInfo.FullName);
            Console.WriteLine("Length of File : " + fileInfo.Length);
            Console.WriteLine("Is Readonly : " + fileInfo.IsReadOnly);
            Console.WriteLine("File is Last Accessed on : " + fileInfo.LastAccessedOn);
            //Deleting File
            Console.WriteLine("Press small y for delete this file");
            try
            {
                char ch = Convert.ToChar(Console.ReadLine());
                if (ch == 'y')
                {
                    if (fileInfo.Exists)
                    {
                        fileInfo.Delete();
                        Console.WriteLine(filePath + " Deleted Successfully");
                    }
                    else
                    {
                        Console.WriteLine("File doesn't exist");
                    }
                }
            }
            catch
            {
            }
        }
    }
}
```

```
        Console.WriteLine("Press Anykey to Exit...");  
    }  
    Console.ReadKey();  
}  
}  
}
```

Output:



```
File Created on : 06-05-2022 18:00:27  
Directory Name : D:\Dotnet  
Name of File : FileInfo1.txt  
Full Name of File : D:\Dotnet\FileInfo1.txt  
Length of File : 22  
Is Readonly : False  
File is Last Accessed on : 06-05-2022 18:00:27  
Press small y for delete this file
```

In the next article, I will discuss [DirectoryInfo Class in C#](#) with Examples. Here, in this article, I try to explain the **FileInfo Class in C#** with Examples. I hope you enjoy this FileInfo Class in C# with Examples article. I would like to have your feedback. Please post your feedback, questions, or comments about this article.